

Even better, you can use the *same* file for all these purposes, without ever being limited to a particular operating system or software—XML works everywhere!

A document written in XML is like a database in itself. And just like a database, its data can be brought into any application that supports it, and then manipulated or presented in any way you choose. So you could make just one XML file and use the content for your Web site or to print a brochure or magazine. You could also share this XML file with someone, who could then use the information for his purposes as well.

Yes, Yes, But What Is It?

XML is, to put it a little simplistically, an extension of HTML (HyperText Markup Language). In fact, its roots are in a format called XHTML—EXtended HTML. All these use the *tag* approach—content is enclosed within markers, called tags, which are then processed by programs called *parsers*, which interpret the meaning of these tags. It looks something like this:

```
<title>This is the Title</title>
```

The first `<title>` tells the parser that the page title begins here. The final `</title>` indicates that this is where the title ends. But this is where the similarity between XML and HTML ends: HTML is designed to display information; XML is designed to *store* it.

While HTML has a defined set of tags, each to denote something specific (like `<head>` for the page header), XML lets you specify your own tags to represent what you want to put in it. For example, if you wanted to put in a phone number, you'd just use:

```
<phone>92724575</phone>
```

And this is where the eXtensible part comes in. You can just keep creating tags to suit your fancy, so there are virtually no limits to what you can do with your XML file. You can even use XML to build a specification for your *own* language!

Rules Are Fun

Of course, we can't have people making XML documents any way they please. In order to make it easier to share and use XML files, the W3C (World Wide Web Consortium) decided to lay down some rules to make sure that all XML files share the same general structure—a tree layout, starting at a node and branching out into sub-nodes.

Rule #1: Thy XML should describe itself

The XML *declaration* is used to define the XML version and its encoding format. It starts the document, and looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
```

Rule #2: Thy XML shall have only one root.

Your XML can contain as many tags as you want, but just like a tree, it should start with only one tag, called the "root".

Rule #3: Thou shalt finish what thou started.

Every tag should have both an opening as well as a closing. This denotes the beginning and end of your data. A closing tag uses a forward slash—the "/" sign—to indicate that the field ends.

Rule # 4: Thou shalt pay attention to case.

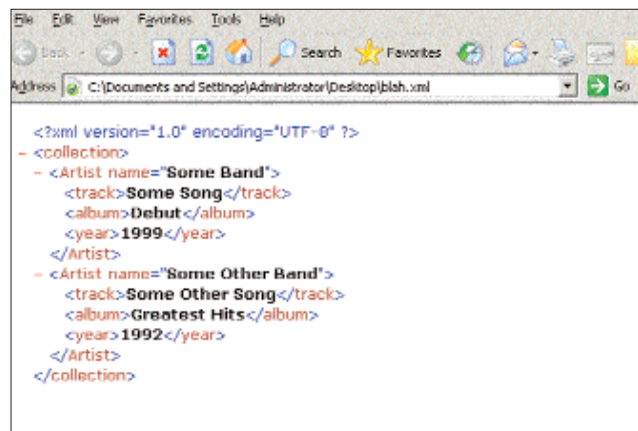
XML tags are case-sensitive, so `<Phone>` is different from `<phone>`. Such details are important.

Let's now apply these rules and make an XML file to store the details of a music collection.

```
<?xml version="1.0" encoding="UTF-8"?>
<collection>
  <Artist name="Some Band">
    <track>Some Song</track>
    <album>Debut</album>
    <year>1999</year>
  </Artist>
  <Artist name="Some Other Band">
    <track>Some Other Song</track>
    <album>Greatest Hits</album>
    <year>1992</year>
  </Artist>
</collection>
```

You can write this in any old text editor, and save the file as "[filename].xml". You will then be able to open it in any Web browser or program that supports XML.

As you can see, tags can even be repeated within the XML. You will also notice that in the "Artist" tag, we've mentioned a *name* as well. These are called attributes, and are usually used to assign unique IDs or names to your tags. You could also have a `<name>` tag under the Artist tag to serve the same purpose. There are no real



The music collection XML as seen in IE

rules when it comes to this, but most who have experience with XML will tell you that you'd rather avoid attributes. The opening tag, closing tag, and the content within are collectively known as an *element*.

So, Now What?

Well, you've made your XML, but what good is it? It still doesn't *do* anything, apart from mocking you from where it sits.

What you do with your XML now really depends on your own expertise and how willing you are to get your hands dirty with some development. When it started out, the purpose of XML was to separate Web pages into the interface - the pretty colours and effects we all see - and the data, the information that is presented using those pretty colours. Indeed, this is the simplest thing we can do with XML.